

Capítulo 3. Metodología y desarrollo del sistema multimodal

3.1 Enfoque metodológico del desarrollo

Esta sección debe abrir el capítulo desde lo académico.

Aquí se explica que el proyecto sigue una metodología aplicada e incremental, orientada a construir un prototipo funcional de simulación de entrevistas laborales para desarrollo de software.

No debe ser muy larga. Su función es justificar cómo se pasó de la revisión bibliográfica al diseño del sistema.

Subtemas sugeridos:

3.1.1 Tipo de investigación

Investigación aplicada, porque busca desarrollar una solución tecnológica concreta.

También puede describirse como descriptiva en la parte de análisis del sistema, porque se documentan procesos, arquitectura, flujos y componentes.

3.1.2 Fases del desarrollo del proyecto

Aquí se puede resumir el proceso:

1. Análisis del problema y revisión de trabajos relacionados.
2. Identificación de funcionalidades requeridas.
3. Diseño del flujo de interacción del candidato.
4. Diseño arquitectónico del sistema.
5. Desarrollo de los módulos funcionales.
6. Integración de servicios de inteligencia artificial.
7. Preparación del sistema para evaluación posterior.

Esto mantiene coherencia con el perfil aprobado, donde el capítulo 3 se plantea como “Metodología y Desarrollo” y no únicamente como metodología de investigación .

3.2 Concepción general del sistema propuesto

Esta sección debe explicar qué es el sistema antes de entrar a partes específicas.

Aquí se puede redactar una visión general:

El sistema propuesto es una plataforma multimodal basada en inteligencia artificial que permite al candidato cargar su CV, generar un perfil técnico, crear una entrevista

adaptada, responder preguntas técnicas, blandas y de código, registrar audio y video, evaluar el desempeño y generar retroalimentación personalizada.

Subtemas sugeridos:

3.2.1 Objetivo funcional del sistema

Explicar que la solución no busca reemplazar procesos reales de contratación, sino ofrecer una herramienta formativa para preparación de entrevistas.

3.2.2 Modalidades consideradas

Aquí se explica la multimodalidad:

- texto del CV;
- texto escrito por el candidato;
- voz de las respuestas;
- video de la entrevista;
- código fuente en preguntas técnicas;
- métricas históricas de desempeño.

3.2.3 Recorrido general del usuario

Aquí sí entra el flujo end-to-end, pero de forma resumida:

1. Registro o inicio de sesión.
2. Carga del CV.
3. Generación del perfil.
4. Configuración de entrevista.
5. Ejecución de entrevista.
6. Procesamiento automático.
7. Consulta de feedback.
8. Historial.

Ese recorrido ya está descrito en los anexos como un flujo secuencial desde autenticación hasta historial de entrevistas .

3.3 Diseño arquitectónico del sistema

Esta sección debe contener la justificación fuerte de arquitectura, incluyendo la división en microservicios.

Aquí no conviene describir todavía cada proceso en detalle. Primero se debe explicar la lógica de organización del sistema.

3.3.1 Estilo arquitectónico seleccionado

Explicar que se adoptó una arquitectura basada en microservicios, con frontend, API Gateway, servicios de dominio, workers asíncronos, base de datos PostgreSQL, Redis y almacenamiento compatible con Google Cloud Storage.

El sistema está organizado con microservicios Node.js, frontend React, API Gateway, workers, PostgreSQL, Redis y emulador local de Google Cloud Storage para archivos .

3.3.2 Criterios para la división en microservicios

Esta sección es clave. Aquí se debe justificar la división.

Los criterios podrían ser:

- 1. Dominio funcional independiente.**

Cada microservicio representa una capacidad central del sistema: autenticación, medios, candidato, entrevista, evaluación o feedback.

- 2. Diferente naturaleza de procesamiento.**

No todos los módulos tienen la misma carga. Autenticación y consulta de perfil son operaciones rápidas; procesamiento de video, evaluación multimodal y generación de feedback son tareas pesadas.

- 3. Necesidad de escalabilidad diferenciada.**

En un despliegue sobre contenedores de Google Cloud, servicios como evaluación o medios pueden requerir más recursos que autenticación o feedback.

- 4. Separación de responsabilidades.**

La carga de archivos no debe mezclarse con generación de preguntas. La evaluación no debe mezclarse con la redacción del feedback. El perfil del candidato no debe depender directamente de la interfaz.

- 5. Facilidad de evolución de modelos de IA.**

Cada componente de IA puede cambiar de modelo, proveedor o estrategia sin afectar todo el sistema.

- 6. Procesamiento asíncrono.**

Las tareas largas se aíslan mediante workers y colas, evitando bloquear la experiencia del usuario.

- 7. Trazabilidad del ciclo de entrevista.**

La división permite seguir el flujo: CV → perfil → entrevista → evaluación → feedback.

3.3.3 Componentes principales de la arquitectura

Aquí se presenta la arquitectura por bloques:

- Frontend.
- API Gateway.
- Auth Service.
- Media Service.
- Candidate Service.
- Interview Service.
- Evaluation Service.
- Feedback Service.
- Workers.
- PostgreSQL.
- Redis.
- Google Cloud Storage o emulador local.
- OpenAI API.
- Judge0.
- FFmpeg.

El anexo de arquitectura general ya identifica estos servicios y sus responsabilidades principales .

3.3.4 Comunicación entre componentes

Explicar:

- El frontend se comunica únicamente con el API Gateway.
- El API Gateway enruta hacia servicios internos.
- Los servicios se comunican mediante HTTP.
- Los procesos pesados se ejecutan mediante colas y workers.
- Los archivos se centralizan en Media Service.
- Los modelos externos de IA no son consumidos por el frontend, sino por servicios especializados.

3.3.5 Estrategia de persistencia

Aquí va la explicación de PostgreSQL con schemas separados.

No hace falta todavía una sección enorme de base de datos. Puede explicarse como parte de la arquitectura:

- Se usa una instancia PostgreSQL compartida.
- Cada servicio mantiene su propio schema.
- La comunicación entre servicios no se hace mediante consultas directas a otros schemas, sino mediante APIs internas.
- Esto conserva separación lógica sin aumentar demasiado la complejidad del prototipo.

El documento de arquitectura indica que cada servicio utiliza un schema lógico separado y que la integración entre dominios se realiza por HTTP .

3.4 Gestión del CV y generación del perfil del candidato

Esta sección ya entra al primer gran bloque funcional.

Aquí se integran flujo, algoritmo, modelo y salida.

3.4.1 Carga y almacenamiento del CV

Explicar:

- El candidato carga un PDF desde la interfaz.
- El archivo se envía al API Gateway.
- Media Service recibe el archivo.
- Se valida y almacena.
- El recurso queda disponible para procesamiento.

3.4.2 Extracción de información desde el CV

Aquí entra Candidate Service.

Explicar:

- Candidate Service recibe el mediaId.
- Valida que sea un PDF en estado disponible.
- Descarga el archivo desde Media Service.
- Envía el documento al modelo de IA.
- Solicita una salida estructurada.

3.4.3 Modelo utilizado para estructuración del perfil

Aquí se habla de OpenAI Responses API, archivo PDF y salida JSON estricta.

No conviene decir simplemente “usa IA”; hay que explicar que el modelo transforma un documento no estructurado en datos evaluables.

El anexo de algoritmos indica que Candidate Service usa OpenAI con un schema JSON estricto para generar CandidateProfile, CandidateTopic, CandidateSubtopic y soft skills .

3.4.4 Perfil técnico generado

Explicar qué produce el sistema:

- datos generales del candidato;
- resumen profesional;
- rol objetivo;
- nivel estimado;
- habilidades técnicas;
- topics;
- subtopics;
- soft skills de catálogo.

3.4.5 Consideraciones y limitaciones

Aquí se pueden mencionar límites reales:

- La calidad depende del contenido del CV.
- El sistema no debe inventar experiencia no sustentada.
- Las soft skills no se extraen directamente del CV, sino que se agregan desde un catálogo.
- Si el CV se actualiza, el perfil puede regenerarse.

3.5 Planificación adaptativa de la entrevista

Esta sección debe explicar cómo el perfil se convierte en una entrevista.

No separar “algoritmo adaptativo” en otra sección general. Aquí es donde tiene sentido.

3.5.1 Selección de temas evaluables

Explicar que el sistema parte de los topics y subtopics generados desde el CV.

Cada subtopic representa una unidad evaluable.

3.5.2 Algoritmo de cobertura y refuerzo

Aquí se explica el algoritmo adaptativo.

El sistema prioriza:

- subtopics no evaluados;
- subtopics con bajo desempeño;
- balance entre habilidades técnicas y blandas;
- reducción de repetición temática.

El anexo de algoritmos describe que la estrategia adaptativa usa métricas como averageScore, usageCount, lastEvaluatedAt y reinforce para balancear cobertura y refuerzo .

3.5.3 Salida del plan adaptativo

Explicar que el resultado es un plan de evaluación, no todavía las preguntas finales.

El plan puede incluir:

- topic;
- subtopic;
- tipo de habilidad;
- nivel esperado;
- prioridad;
- razón de selección.

3.5.4 Relación con entrevistas futuras

Explicar que después de cada entrevista, los resultados alimentan el desempeño del candidato, permitiendo que el sistema refuerce debilidades en futuras simulaciones.

3.6 Generación de preguntas de entrevista

Esta sección debe continuar naturalmente: ya existe un plan, ahora se generan preguntas.

3.6.1 Generación de preguntas con inteligencia artificial

Explicar:

- Interview Service recibe el plan adaptativo.

- Por cada subtopic solicita una pregunta.
- El modelo genera preguntas técnicas, blandas o de código.
- Las preguntas se adaptan al rol, nivel y subtopic.

3.6.2 Modelo utilizado para generación

Aquí se habla del modelo configurado para generación de preguntas, por ejemplo OpenAI con salida estructurada.

El anexo de decisiones tecnológicas indica que Interview Service usa OpenAI para crear preguntas concretas por subtopic definido en el plan adaptativo .

3.6.3 Control de repetición mediante embeddings

Esta parte entra aquí, no como sección aislada.

Explicar:

- Cada pregunta generada se convierte en embedding.
- Se compara contra preguntas anteriores del mismo usuario.
- Se calcula similitud coseno.
- Si la similitud supera un umbral, se genera una nueva pregunta.
- Si se agotan intentos, se conserva la menos similar.

El anexo de algoritmos indica que este control se realiza con embeddings y similitud coseno para reducir preguntas semánticamente repetidas .

3.6.4 Persistencia de la entrevista creada

Explicar que una vez aceptadas las preguntas, la entrevista queda guardada en estado creado y puede iniciar la simulación.

3.7 Ejecución multimodal de la entrevista

Esta sección debe explicar lo que ocurre durante la entrevista desde el usuario y desde el sistema.

3.7.1 Inicio de la entrevista

Explicar:

- El usuario inicia la entrevista.
- Se activan cámara y micrófono.
- La entrevista pasa a estado en progreso.

- El frontend muestra preguntas secuenciales.

3.7.2 Registro de respuestas

Explicar las modalidades de respuesta:

- respuesta oral;
- respuesta escrita complementaria;
- código en preguntas de programación;
- video continuo de la sesión.

3.7.3 Registro temporal por pregunta

Esta parte es muy importante.

Explicar:

- No se graba un video por pregunta.
- Se graba un solo video de toda la entrevista.
- El frontend registra el inicio y fin de cada pregunta.
- Esos timestamps permiten segmentar después el audio y video.

El anexo de algoritmos indica que el sistema registra videoStartMs y videoEndMs para vincular cada respuesta con un segmento específico del video completo .

3.7.4 Finalización y envío a evaluación

Explicar:

- El usuario finaliza la entrevista.
- Se sube el video.
- Se guardan respuestas, código y timestamps.
- Interview Service marca la entrevista como finalizada.
- Se solicita la evaluación automática.

3.8 Procesamiento multimedia de la entrevista

Esta sección debe concentrarse en video/audio, porque es un bloque con lógica propia.

3.8.1 Almacenamiento y normalización de video

Explicar que Media Service recibe el video, lo registra y lo envía a procesamiento.

3.8.2 Procesamiento con FFmpeg

Explicar:

- conversión a formato optimizado;
- ajuste de resolución;
- límite de FPS;
- codificación H.264/AAC;
- obtención de duración con ffprobe.

El documento de algoritmos indica que Media Worker usa FFmpeg para escalar, limitar FPS, codificar y marcar el video como disponible .

3.8.3 Segmentación para evaluación

Explicar que Evaluation Service no analiza todo el video como una sola unidad, sino que utiliza timestamps para extraer segmentos por pregunta.

3.8.4 Ventajas del enfoque

- Evita subir varios videos.
- Reduce complejidad en frontend.
- Mantiene trazabilidad por pregunta.
- Permite reproducir fragmentos específicos en el feedback.

3.9 Evaluación multimodal de respuestas

Esta puede ser una de las secciones más extensas del capítulo.

Debe integrar semántica, audio, video y código.

3.9.1 Orquestación de la evaluación

Explicar:

- Evaluation Service recibe la solicitud.
- Se crea un job.
- Evaluation Worker procesa en segundo plano.
- Consulta el payload completo de entrevista.
- Descarga el video.
- Segmenta audio/video.
- Evalúa cada pregunta.

El anexo de diagramas de secuencia muestra que, al finalizar la entrevista, se crea una evaluación, el worker obtiene el payload, descarga el video, segmenta y guarda resultados .

3.9.2 Evaluación semántica

Explicar:

- Se usa transcripción, respuesta escrita y código cuando existe.
- El modelo evalúa coherencia, precisión técnica, claridad, profundidad y relevancia.
- Devuelve puntuaciones y justificación.

El anexo de algoritmos indica que el análisis semántico devuelve scores de coherencia, precisión técnica, claridad, profundidad, relevancia y recomendaciones .

3.9.3 Transcripción de audio

Explicar que el audio del segmento se transcribe para convertir la respuesta oral en texto analizable.

Aquí se puede mencionar el modelo de transcripción configurado, como Whisper, según las decisiones tecnológicas del dominio .

3.9.4 Análisis básico de audio

Aquí hay que ser cuidadosos.

No llamarlo “prosodia avanzada” si no lo hace.

Redacción sugerida: análisis básico de fluidez oral.

Incluye:

- cantidad de palabras;
- duración;
- palabras por minuto;
- penalización por respuestas muy cortas;
- penalización por ritmo demasiado bajo o demasiado alto.

El documento de algoritmos aclara que el MVP no estima pausas reales ni características acústicas profundas .

3.9.5 Análisis visual del video

Explicar:

- validación del segmento;
- revisión de metadata;
- extracción de frames cuando sea posible;
- análisis de señales observables como encuadre, presencia, orientación y atención aparente.

También conviene aclarar que no se diagnostican emociones, salud mental ni rasgos personales.

3.9.6 Evaluación de código

Explicar:

- Para preguntas de código se usa Judge0 cuando está configurado.
- El sistema analiza ejecución, errores y estado del código.
- La evaluación puede complementarse con análisis semántico.

También se debe mencionar una limitación: si no existen casos de prueba o salida esperada, la evaluación automática de código queda parcialmente limitada.

3.9.7 Cálculo de puntajes

Explicar que el sistema calcula puntajes por pregunta y puntajes globales.

Se puede mencionar que los pesos dependen del tipo de pregunta:

- técnica;
- soft skill;
- código.

No hace falta poner demasiadas fórmulas, pero sí explicar que la agregación es ponderada y configurable.

3.10 Generación de retroalimentación personalizada

Esta sección debe cerrar el ciclo funcional.

3.10.1 Activación del proceso de feedback

Explicar que cuando la evaluación termina, Evaluation Service notifica a Feedback Service.

3.10.2 Construcción del reporte

Explicar que Feedback Service consulta:

- evaluación global;
- evaluación por pregunta;
- fortalezas;
- debilidades;
- recomendaciones;
- contexto del candidato si está disponible.

3.10.3 Modelo utilizado para redacción del feedback

Aquí entra OpenAI como generador del reporte final.

Pero hay que aclarar que el modelo **no recalcula los scores**, sino que redacta el feedback con base en resultados ya generados.

El anexo de algoritmos indica que Feedback Service genera el reporte sin modificar puntajes y puede usar un fallback heurístico si el modelo no está disponible .

3.10.4 Estructura del feedback entregado al usuario

Puede incluir:

- score global;
- resumen ejecutivo;
- fortalezas;
- áreas de mejora;
- recomendaciones;
- análisis por pregunta;
- resultados técnicos;
- resultados de habilidades blandas;
- resultados de código;
- acceso a segmentos de video.

3.10.5 Consulta de historial

Aquí puede cerrarse el flujo con historial de entrevistas y revisión de reportes anteriores.

3.11 Despliegue previsto e infraestructura de ejecución

Esta sección reemplaza al antiguo 3.10.

No debe ser “cómo hicimos el repo”, sino cómo se concibe la ejecución del sistema.

3.11.1 Contenerización de servicios

Explicar que cada servicio está pensado para ejecutarse como contenedor independiente.

Esto se conecta con el despliegue futuro en Google Cloud.

3.11.2 Despliegue sobre servicios administrados de Google Cloud

Aquí se puede plantear de forma general:

- servicios backend desplegados como contenedores;
- almacenamiento de archivos en Google Cloud Storage;
- base de datos administrada o equivalente;
- variables de entorno para claves y modelos;
- escalabilidad independiente por servicio.

No hay que comprometerse con detalles que todavía no estén implementados, pero sí dejar claro que la arquitectura está preparada para ese modelo.

3.11.3 Entorno local de desarrollo

Aquí sí se menciona el monorepo y Docker Compose, pero como soporte de desarrollo, no como implementación final.

Explicar:

- el monorepo facilitó el desarrollo coordinado;
- Docker Compose permitió levantar servicios localmente;
- fake-gcs-server permitió emular Google Cloud Storage;
- Redis y PostgreSQL se usaron como infraestructura local.

3.11.4 Diferencia entre entorno local y despliegue final

Esta parte es importante.

Puedes decir:

El entorno local reproduce las dependencias principales para facilitar desarrollo y pruebas, pero el despliegue final se orienta a contenedores en Google Cloud, manteniendo la misma separación por servicios.

3.12 Consideraciones éticas y limitaciones técnicas del diseño

Esta sección sí conviene mantener, porque el sistema evalúa personas con IA.

No es validación; son consideraciones de diseño.

3.12.1 Propósito formativo del sistema

Aclarar que el sistema es de entrenamiento, no de selección laboral real.

3.12.2 Privacidad de datos y archivos

Mencionar CV, video, audio, respuestas y código.

3.12.3 Limitaciones de modelos de IA

Riesgo de errores, variabilidad, dependencia del proveedor y necesidad de interpretación humana.

3.12.4 Limitaciones del análisis visual y de audio

Aclarar que las métricas no deben usarse como diagnóstico psicológico ni como juicio absoluto del candidato.

3.12.5 Transparencia del feedback

El sistema debe presentar recomendaciones comprensibles, no solamente puntajes.

Índice revisado para llevar al tutor

Esta sería la versión más limpia:

Capítulo 3. Metodología y desarrollo del sistema multimodal

3.1 Enfoque metodológico del desarrollo

3.1.1 Tipo de investigación

3.1.2 Fases del desarrollo del proyecto

3.2 Concepción general del sistema propuesto

3.2.1 Objetivo funcional del sistema

3.2.2 Modalidades consideradas

3.2.3 Recorrido general del usuario

3.3 Diseño arquitectónico del sistema

3.3.1 Estilo arquitectónico seleccionado

3.3.2 Criterios para la división en microservicios

3.3.3 Componentes principales de la arquitectura

3.3.4 Comunicación entre componentes

3.3.5 Estrategia de persistencia

3.4 Gestión del CV y generación del perfil del candidato

3.4.1 Carga y almacenamiento del CV

- 3.4.2 Extracción de información desde el CV
- 3.4.3 Modelo utilizado para estructuración del perfil
- 3.4.4 Perfil técnico generado
- 3.4.5 Consideraciones y limitaciones

3.5 Planificación adaptativa de la entrevista

- 3.5.1 Selección de temas evaluables
- 3.5.2 Algoritmo de cobertura y refuerzo
- 3.5.3 Salida del plan adaptativo
- 3.5.4 Relación con entrevistas futuras

3.6 Generación de preguntas de entrevista

- 3.6.1 Generación de preguntas con inteligencia artificial
- 3.6.2 Modelo utilizado para generación
- 3.6.3 Control de repetición mediante embeddings
- 3.6.4 Persistencia de la entrevista creada

3.7 Ejecución multimodal de la entrevista

- 3.7.1 Inicio de la entrevista
- 3.7.2 Registro de respuestas
- 3.7.3 Registro temporal por pregunta
- 3.7.4 Finalización y envío a evaluación

3.8 Procesamiento multimedia de la entrevista

- 3.8.1 Almacenamiento y normalización de video
- 3.8.2 Procesamiento con FFmpeg
- 3.8.3 Segmentación para evaluación
- 3.8.4 Ventajas del enfoque

3.9 Evaluación multimodal de respuestas

- 3.9.1 Orquestación de la evaluación
- 3.9.2 Evaluación semántica
- 3.9.3 Transcripción de audio
- 3.9.4 Análisis básico de audio
- 3.9.5 Análisis visual del video
- 3.9.6 Evaluación de código
- 3.9.7 Cálculo de puntajes

3.10 Generación de retroalimentación personalizada

- 3.10.1 Activación del proceso de feedback
- 3.10.2 Construcción del reporte
- 3.10.3 Modelo utilizado para redacción del feedback
- 3.10.4 Estructura del feedback entregado al usuario
- 3.10.5 Consulta de historial

3.11 Despliegue previsto e infraestructura de ejecución

3.11.1 Contenerización de servicios

3.11.2 Despliegue sobre servicios administrados de Google Cloud

3.11.3 Entorno local de desarrollo

3.11.4 Diferencia entre entorno local y despliegue final

3.12 Consideraciones éticas y limitaciones técnicas del diseño

3.12.1 Propósito formativo del sistema

3.12.2 Privacidad de datos y archivos

3.12.3 Limitaciones de modelos de IA

3.12.4 Limitaciones del análisis visual y de audio

3.12.5 Transparencia del feedback